

Application of Iterative and Decision-Making Constructs to Shapes and Physics using Modular Programming in C

Name: Dhruv Pandey

SAP ID: 590037660

Batch: 15

MODULAR C PROGRAMMING REPORT

1. Problem Overview

Develop a menu-driven modular C project for geometrical measurements and physical properties of a sphere, cylinder, cone, and cuboid. The project uses fundamental data types, call-by-value functions, selection statements, and loops. It does not use recursion, arrays, pointer variables/parameters, structures, unions, enumerations, typedef, or functions from math.h.

Project Organization

Module	Responsibility
main.c	Displays menus, obtains common inputs, validates values, calls functions, and displays results.
geometry.c	Calculates surface area, volume, base area, and cone slant height.
physics.c	Obtains solid type/dimensions for physics operations and calculates mass, weight, energy, pressure, buoyant force, and buoyancy behavior.
math_utils.c	Calculates square root iteratively using Newton-Raphson.
solid.h	Header guard, symbolic constants, and external function declarations.

2. Algorithms

Surface Area

Step 1: Input/select solid type and validate the choice.

Step 2: Obtain required dimensions and validate each dimension (> 0).

Step 3: Sphere: return $4 * \text{PI} * r * r$.

Step 4: Cylinder: return $2 * \text{PI} * r * (r + h)$.

Step 5: Cone: calculate slant height using `calculate_square_root( $r*r + h*h$ )`, then return $\text{PI} * r * (r + \text{slant_height})$.

Step 6: Cuboid: return $2 * (l*b + b*h + l*h)$.

Step 7: Display area in square metres.

Volume

Step 1: Select and validate solid type.

Step 2: Read and validate dimensions (> 0).

Step 3: Call `sphere_volume`, `cylinder_volume`, `cone_volume`, or `cuboid_volume`.

Step 4: Display the returned volume in cubic metres.

Base Area

Step 1: Select a solid.

Step 2: If sphere is selected, display that ideal geometrical base area is not defined and return to the menu.

Step 3: For cylinder/cone, validate radius and call `circular_base_area()`.

Step 4: For cuboid, validate length and breadth and call `cuboid_base_area()`.

Step 5: Display area in square metres.

Mass

Step 1: Input density and validate density > 0 .

Step 2: `calculate_mass()` obtains and validates the solid type and its dimensions.

Step 3: Calculate volume by calling the appropriate geometry function.

Step 4: Return density * volume.

Step 5: Display mass in kilograms.

Weight

Step 1: Call calculate_mass() to obtain mass.

Step 2: Return mass * GRAVITY.

Step 3: Display weight in newtons.

Gravitational Potential Energy

Step 1: Input elevation and validate elevation ≥ 0 .

Step 2: Call calculate_mass() to obtain mass.

Step 3: Return mass * GRAVITY * elevation.

Step 4: Display energy in joules.

Pressure on Base

Step 1: Input density and validate density > 0 .

Step 2: Obtain solid type; allow only cylinder, cone, or cuboid.

Step 3: Validate dimensions.

Step 4: Obtain volume from the appropriate volume function.

Step 5: Obtain contact area from circular_base_area() or cuboid_base_area().

Step 6: Return (density * volume * GRAVITY) / contact_area.

Step 7: Display pressure in pascals.

Buoyant Force

Step 1: Input fluid density and validate fluid_density > 0 .

Step 2: Obtain and validate solid type and dimensions.

Step 3: Calculate volume with the appropriate geometry function.

Step 4: Return fluid_density * volume * GRAVITY.

Step 5: Display force in newtons.

Buoyancy Behavior

Step 1: Input solid density and fluid density; validate both > 0 .

Step 2: If solid_density $>$ fluid_density, return SINKS.

Step 3: If solid_density $<$ fluid_density, return FLOATS.

Step 4: Otherwise return SUSPENDED.

Step 5: Display the corresponding message.

Iterative Square Root

Step 1: If number < 0 , return -1.

Step 2: If number $== 0$, return 0.

Step 3: Set old_value = number.

Step 4: Repeat: new_value = $0.5 * (\text{old_value} + \text{number} / \text{old_value})$.

Step 5: $\text{difference} = \text{new_value} - \text{old_value}$; if negative, make it positive.

Step 6: Set $\text{old_value} = \text{new_value}$.

Step 7: Continue while $\text{difference} \geq \text{TOLERANCE}$.

Step 8: Return new_value .

3. Test-Case Table

TC	Operation	Input	Expected Output / Behavior	Type
1	Surface area - Sphere	r=1	12.5664 m ²	Valid
2	Volume - Cylinder	r=1, h=2	6.2832 m ³	Valid
3	Surface area - Cone	r=3, h=4	75.3984 m ²	Valid
4	Volume - Cuboid	l=2, b=3, h=4	24.0000 m ³	Valid
5	Base area - Cylinder	r=2	12.5664 m ²	Valid
6	Base area - Sphere	Sphere selected	Message: base area not defined	Boundary/unsupported
7	Mass - Sphere	density=1000, r=1	4188.8 kg	Valid
8	Potential energy	elevation=0	0 J	Boundary
9	Buoyancy behavior	solid=500, fluid=1000	FLOATS	Valid
10	Buoyancy behavior	solid=1000, fluid=1000	SUSPENDED	Boundary
11	Buoyancy behavior	solid=1200, fluid=1000	SINKS	Valid
12	Dimension validation	radius=0	Error; request radius again	Invalid
13	Density validation	density=-10	Error; request density again	Invalid
14	Elevation validation	elevation=-1	Error; request elevation again	Invalid
15	Solid validation	solid choice=5	Error; request choice again	Invalid
16	Menu validation	menu choice=0	Error; request valid menu choice	Invalid

4. Source Code

solid.h

```
#ifndef SOLID_H
#define SOLID_H

#define PI 3.1416
#define GRAVITY 9.8
#define TOLERANCE 0.000001

#define SPHERE 1
#define CYLINDER 2
#define CONE 3
#define CUBOID 4

#define FLOATS -1
#define SUSPENDED 0
#define SINKS 1

double calculate_square_root(double number);

double sphere_area(double radius);
double sphere_volume(double radius);
double cylinder_area(double radius, double height);
double cylinder_volume(double radius, double height);
double cone_area(double radius, double height);
double cone_volume(double radius, double height);
double cuboid_area(double length, double breadth, double height);
double cuboid_volume(double length, double breadth, double height);
double circular_base_area(double radius);
double cuboid_base_area(double length, double breadth);

double calculate_mass(double density);
double calculate_weight(void);
double potential_energy(double elevation);
double calculate_pressure(double density);
double buoyant_force(double fluid_density);
int determine_buoyancy(double solid_density, double fluid_density);

#endif
```

geometry.c

```
#include "solid.h"

double sphere_area(double r) { return 4 * PI * r * r; }
double sphere_volume(double r) { return (4.0 / 3.0) * PI * r * r * r; }

double cylinder_area(double r, double h) {
    return 2 * PI * r * (r + h);
}
double cylinder_volume(double r, double h) {
    return PI * r * r * h;
}

double cone_area(double r, double h) {
    double slant_height = calculate_square_root(r * r + h * h);
    return PI * r * (r + slant_height);
}
double cone_volume(double r, double h) {
    return (1.0 / 3.0) * PI * r * r * h;
}

double cuboid_area(double l, double b, double h) {
    return 2 * (l * b + b * h + l * h);
}
double cuboid_volume(double l, double b, double h) {
    return l * b * h;
}
double circular_base_area(double r) { return PI * r * r; }
double cuboid_base_area(double l, double b) { return l * b; }
```

physics.c

```
#include <stdio.h>
#include "solid.h"
```

```

static int get_solid(void) {
    int solid;
    do {
        printf("1.Sphere 2.Cylinder 3.Cone 4.Cuboid: ");
        scanf("%d", &solid);
        if (solid < SPHERE || solid > CUBOID)
            printf("Invalid solid choice.\n");
    } while (solid < SPHERE || solid > CUBOID);
    return solid;
}

static double positive_value(const char *prompt) {
    double x;
    do {
        printf("%s", prompt);
        scanf("%lf", &x);
        if (x <= 0) printf("Value must be greater than zero.\n");
    } while (x <= 0);
    return x;
}

static double get_volume(int solid) {
    double r, h, l, b;
    if (solid == SPHERE) {
        r = positive_value("Radius (m): ");
        return sphere_volume(r);
    }
    if (solid == CYLINDER) {
        r = positive_value("Radius (m): ");
        h = positive_value("Height (m): ");
        return cylinder_volume(r, h);
    }
    if (solid == CONE) {
        r = positive_value("Radius (m): ");
        h = positive_value("Height (m): ");
        return cone_volume(r, h);
    }
    l = positive_value("Length (m): ");
    b = positive_value("Breadth (m): ");
    h = positive_value("Height (m): ");
    return cuboid_volume(l, b, h);
}

double calculate_mass(double density) {
    int solid = get_solid();
    return density * get_volume(solid);
}

double calculate_weight(void) {
    double density = positive_value("Density (kg/m^3): ");
    return calculate_mass(density) * GRAVITY;
}

double potential_energy(double elevation) {
    double density = positive_value("Density (kg/m^3): ");
    return calculate_mass(density) * GRAVITY * elevation;
}

double calculate_pressure(double density) {
    int solid;
    double r, h, l, b, volume, area;
    do {
        solid = get_solid();
        if (solid == SPHERE)
            printf("Pressure on an ideal sphere base is not defined.\n");
    } while (solid == SPHERE);

    if (solid == CYLINDER || solid == CONE) {
        r = positive_value("Radius (m): ");
        h = positive_value("Height (m): ");
        area = circular_base_area(r);
        if (solid == CYLINDER) volume = cylinder_volume(r, h);
        else volume = cone_volume(r, h);
    } else {
        l = positive_value("Length (m): ");
        b = positive_value("Breadth (m): ");
        h = positive_value("Height (m): ");
        area = cuboid_base_area(l, b);
        volume = cuboid_volume(l, b, h);
    }
}

```

```

    }
    return (density * volume * GRAVITY) / area;
}

double buoyant_force(double fluid_density) {
    int solid = get_solid();
    return fluid_density * get_volume(solid) * GRAVITY;
}

int determine_buoyancy(double solid_density, double fluid_density) {
    if (solid_density > fluid_density) return SINKS;
    if (solid_density < fluid_density) return FLOATS;
    return SUSPENDED;
}

```

math_utils.c

```

#include "solid.h"

double calculate_square_root(double number) {
    double old_value, new_value, difference;
    if (number < 0) return -1;
    if (number == 0) return 0;

    old_value = number;
    do {
        new_value = 0.5 * (old_value + number / old_value);
        difference = new_value - old_value;
        if (difference < 0) difference = -difference;
        old_value = new_value;
    } while (difference >= TOLERANCE);

    return new_value;
}

```

main.c

```

#include <stdio.h>
#include "solid.h"

double read_positive(const char *prompt) {
    double x;
    do {
        printf("%s", prompt);
        scanf("%lf", &x);
        if (x <= 0) printf("Value must be greater than zero.\n");
    } while (x <= 0);
    return x;
}

int read_solid(void) {
    int s;
    do {
        printf("1.Sphere 2.Cylinder 3.Cone 4.Cuboid: ");
        scanf("%d", &s);
        if (s < 1 || s > 4) printf("Invalid solid choice.\n");
    } while (s < 1 || s > 4);
    return s;
}

int main(void) {
    int choice, solid, result;
    double r, h, l, b, density, fluid_density, elevation, value;

    do {
        printf("\n1.Surface area\n2.Volume\n3.Base Area\n4.Mass\n");
        printf("5.Weight\n6.Gravitational potential energy\n");
        printf("7.Pressure on the base\n8.Buoyant force\n");
        printf("9.Buoyancy behavior\n10.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        if (choice < 1 || choice > 10) {
            printf("Invalid menu choice.\n");
            continue;
        }

        switch (choice) {
            case 1:

```

```

case 2:
    solid = read_solid();
    if (solid == SPHERE) {
        r = read_positive("Radius (m): ");
        value = (choice == 1) ? sphere_area(r) : sphere_volume(r);
    } else if (solid == CYLINDER) {
        r = read_positive("Radius (m): ");
        h = read_positive("Height (m): ");
        value = (choice == 1) ? cylinder_area(r,h) : cylinder_volume(r,h);
    } else if (solid == CONE) {
        r = read_positive("Radius (m): ");
        h = read_positive("Height (m): ");
        value = (choice == 1) ? cone_area(r,h) : cone_volume(r,h);
    } else {
        l = read_positive("Length (m): ");
        b = read_positive("Breadth (m): ");
        h = read_positive("Height (m): ");
        value = (choice == 1) ? cuboid_area(l,b,h) : cuboid_volume(l,b,h);
    }
    printf(choice == 1 ? "Area = %.4f m^2\n" : "Volume = %.4f m^3\n", value);
    break;

case 3:
    solid = read_solid();
    if (solid == SPHERE) {
        printf("Base area of a sphere is not defined.\n");
        break;
    }
    if (solid == CYLINDER || solid == CONE) {
        r = read_positive("Radius (m): ");
        value = circular_base_area(r);
    } else {
        l = read_positive("Length (m): ");
        b = read_positive("Breadth (m): ");
        value = cuboid_base_area(l,b);
    }
    printf("Base area = %.4f m^2\n", value);
    break;

case 4:
    density = read_positive("Density (kg/m^3): ");
    printf("Mass = %.4f kg\n", calculate_mass(density));
    break;

case 5:
    printf("Weight = %.4f N\n", calculate_weight());
    break;

case 6:
    do {
        printf("Elevation (m): ");
        scanf("%lf", &elevation);
        if (elevation < 0) printf("Elevation cannot be negative.\n");
    } while (elevation < 0);
    printf("Potential energy = %.4f J\n", potential_energy(elevation));
    break;

case 7:
    density = read_positive("Density (kg/m^3): ");
    printf("Pressure = %.4f Pa\n", calculate_pressure(density));
    break;

case 8:
    fluid_density = read_positive("Fluid density (kg/m^3): ");
    printf("Buoyant force = %.4f N\n", buoyant_force(fluid_density));
    break;

case 9:
    density = read_positive("Solid density (kg/m^3): ");
    fluid_density = read_positive("Fluid density (kg/m^3): ");
    result = determine_buoyancy(density, fluid_density);
    if (result == FLOATS) printf("The solid FLOATS.\n");
    else if (result == SUSPENDED) printf("The solid remains SUSPENDED.\n");
    else printf("The solid SINKS.\n");
    break;

case 10:
    printf("Exiting program.\n");
    break;

```

```
    }  
} while (choice != 10);  
return 0;  
}
```

5. Compilation and Execution

Compile all modules together using:

```
cc main.c geometry.c physics.c math_utils.c -o solids
```

Run the executable using:

```
./solids
```

6. Testing Result

The project is designed to validate solid choices, dimensions, densities, elevation, and menu choices immediately after input. The prepared test cases cover valid, invalid, boundary, and unsupported cases, including zero elevation, equal densities, invalid dimensions, and the undefined ideal base/contact area of a sphere.

Expected conclusion: After compiling and executing the project with the prepared test cases, the displayed results should match the expected outputs and all invalid inputs should trigger an appropriate error message followed by re-entry.